

SOP REPORT

Mohammed Ashraf and Chaitanya Joshi

[Basics of the switch](#)

[gRPC](#)

[Working with table entries](#)

- [1. Example starter program](#)
- [2. Adding table entries](#)
- [3. Delete table entries](#)
- [4. Update table entries](#)
- [5. Read \(dump\) current table entries](#)
- [6. Running the starter program](#)
- [7. An example](#)

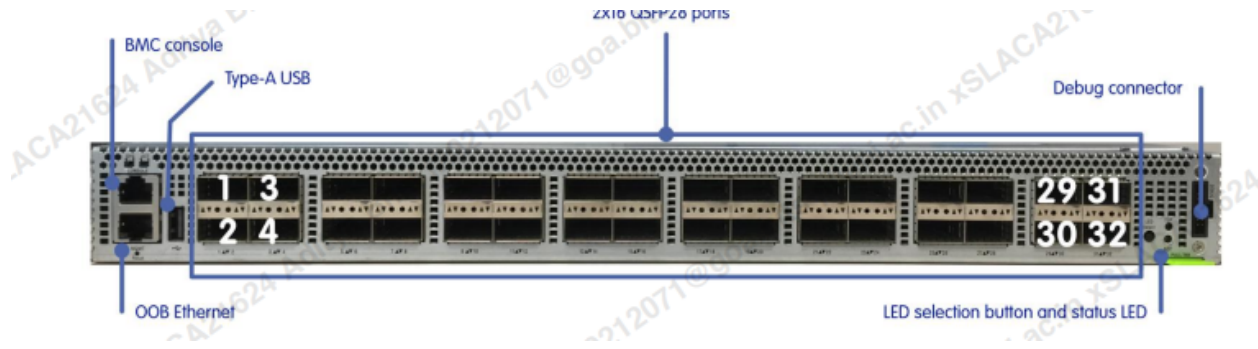
[Packet Redirection using CPU Ports](#)

[Self learning switch](#)

[PCIe Express CPU port](#)

[Future work](#)

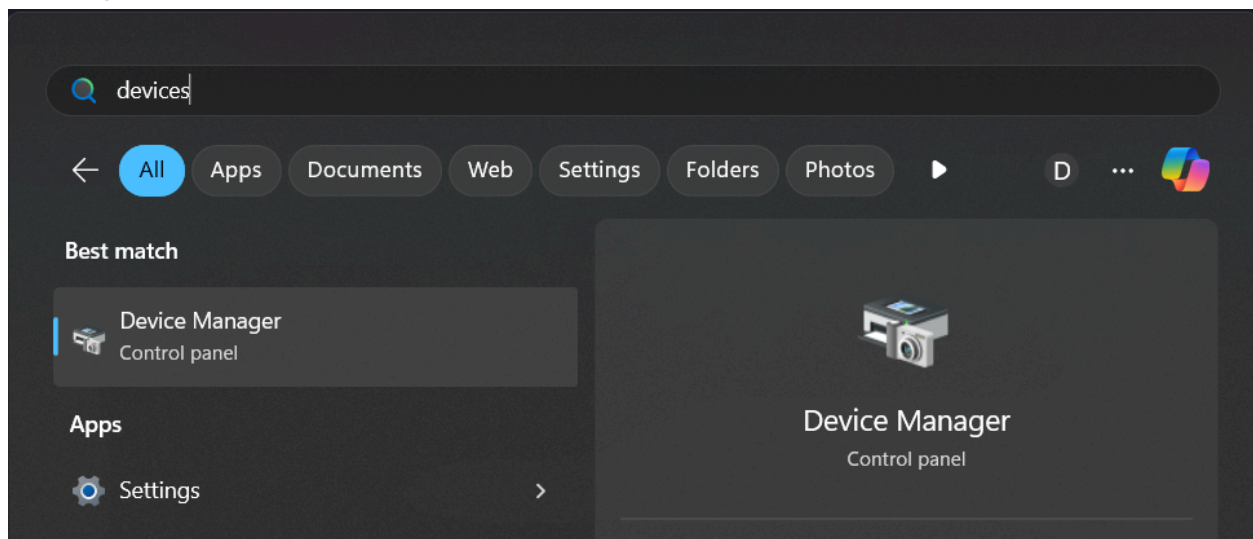
Basics of the switch



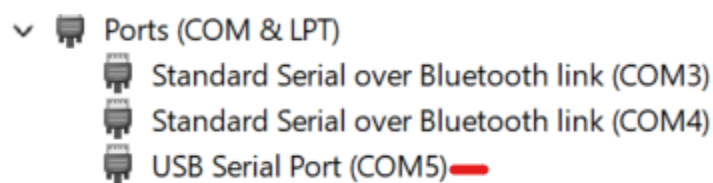
General control of the system is provided by the baseboard management controller CPU (BMC). This is what you first see when you connect to the switch using the serial port (BMC Console).

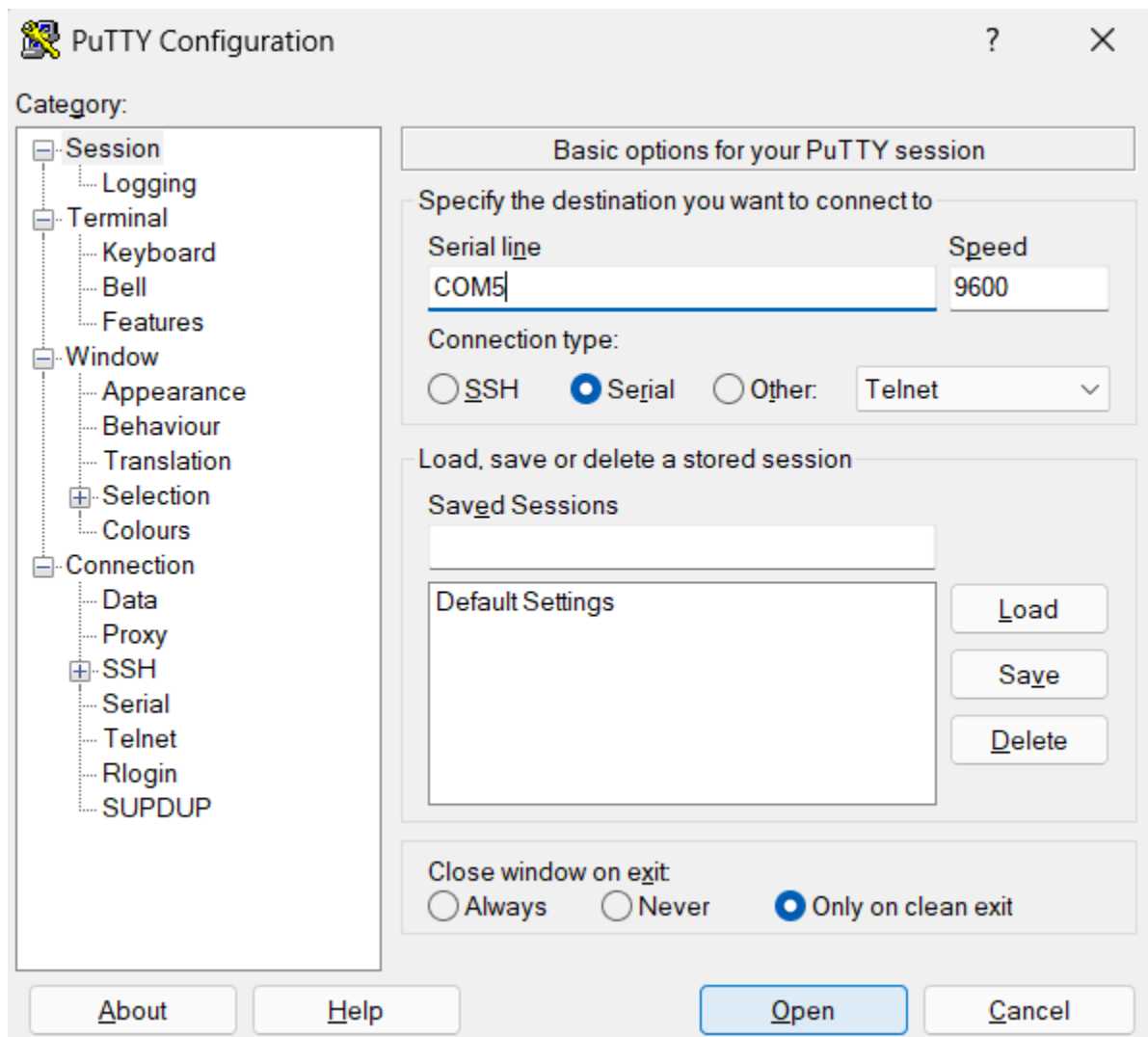
To connect to the switch via serial/console port, use a terminal emulator that supports serial communication, like PuTTY

When you connect the BMC Console port to your laptop, you'll find a USB serial port with a serial line number under the ports section in the device manager.



Use the serial line number to open a serial connection to the switch (if the switch os is powered on you will enter the ubuntu os directly, else you will land on the BMC)





From BMC you power on the switch os (if its not already on) which holds the Barefoot SDE using a script, **wedge_power.sh**. Once on you can enter the switch os using **sol.sh**. Use **ctrl + x** to exit the sol session back to the bmc.

ssh into the switch os directly from the antserver using the command
`ssh -p 9999 developer@switch`

Setting up the reverse ssh tunnel onto the switch os directly bypassing bmc

```
developer@switch0:~$ ps aux | grep ssh
root      920    0.0  0.1 15436 9308 ?        Ss   Jan25   0:00 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
develop+  94375   0.0  0.0 11696 4020 ?        Ss   Apr09   0:00 ssh -R 0.0.0.0:9999:localhost:22 -fN f20212909@antserver.csis.bits-go.a
root      99738   1.1  0.1 17184 10940 ?       Ss   08:45   0:00 sshd: developer [priv]
develop+  99798   0.0  0.1 17316 8168 ?        S    08:45   0:00 sshd: developer@pts/0
develop+  99809   0.0  0.0  4028 2096 pts/0    S+   08:45   0:00 grep --color=auto ssh
```

The command used to set up the reverse tunnel.

`-R 0.0.0.0:9999:localhost:22`

- This sets up a **reverse tunnel**.
- It tells the server to listen on **port 9999** on **all interfaces (0.0.0.0)** of the server.
- Any connection made to the server on port **9999** will be **forwarded back** through the tunnel to **port 22** (SSH) on the **localhost of the machine running this command** (your switch).

Using gRPC to communicate with the barefoot runtime control plane

Overview of BF Runtime

BF Runtime serves as the control plane API for Tofino programmable switches, providing a standard interface to interact with P4 programs running on the device. When using gRPC with BF-RT, a protobuf specification defines the API structure, generating Python bindings that enable client applications to communicate with the C++ gRPC server running on the switch.

gRPC Connection Process

Establishing a connection to the BF Runtime server requires three essential steps:

- A. Channel Establishment: Opening an insecure channel to port 50052 on the host running the gRPC server
- B. Subscription: Sending a "subscribe" message through the RPC StreamChannel to establish a bidirectional communication stream
- C. Pipeline Configuration: Sending a "SetForwardingPipelineConfigRequest" message with client ID, device ID, and P4 program name.

All these 3 steps are handled by just one python command

Required packages

```
import grpc  
import bfrt_grpc.bfruntime_pb2 as bfruntime_pb2  
import bfrt_grpc.client as bfrt_client
```

Command to connect the bfrt client (your python program) to the BFRT server (listening on localhost port 50052)

```
interface = bfrt_client.ClientInterface(  
grpc_addr = 'localhost:50052',  
    client_id = 0,  
    device_id = 0)  
print('Connected to BF Runtime Server')
```

Then you can use this interface to work with the bfrt server.

```
# Get the information about the running program  
bfrt_info = interface.bfrt_info_get()  
print('The target runs the program ', bfrt_info.p4_name_get())  
  
# Establish that you are using this program on the given  
connection  
interface.bind_pipeline_config(bfrt_info.p4_name_get())
```

Working with the table entries

The BF Runtime (BFRT) system provides a programmatic interface for runtime control of P4-defined network devices. To interact with match-action tables in a P4 program, you should modify the `bfrt_starter.py` file, which serves as the bridge between the Python control environment and the P4 device.

Basic structure of `bfrt_starter.py`:

ref: https://github.com/ashhraaffff/p4-experiments/blob/main/gRPC/bfrt_starter.py

```
#!/usr/bin/env python

from __future__ import print_function

import os
import sys
import pdb
import socket
import struct

SDE_INSTALL = os.environ['SDE_INSTALL']
SDE_PYTHON_310 = os.path.join(SDE_INSTALL, 'lib', 'python3.10', 'site-packages')

sys.path.append(SDE_PYTHON_310)
sys.path.append(os.path.join(SDE_PYTHON_310, 'tofino'))

import grpc
import bfrt_grpc.bfruntime_pb2 as bfruntime_pb2
import bfrt_grpc.client as bfrt_client

#
# Connect to the BF Runtime Server
#
interface = bfrt_client.ClientInterface(
    grpc_addr = 'localhost:50052',
    client_id = 0,
    device_id = 0)
print('Connected to BF Runtime Server')

#
# Get the information about the running program
#
bfrt_info = interface.bfrt_info_get()
print('The target runs the program ', bfrt_info.p4_name_get())

#
# Establish that you are using this program on the given connection
#
interface.bind_pipeline_config(bfrt_info.p4_name_get())

##### You can now use BFRT CLIENT #####
table1 = bfrt_info.table_get("Ingress.ipv4_host")
##### FINALLY #####

# If you use SDE prior to 9.4.0, uncomment the line below
# interface._tear_down_stream()

print("The End")
```

Tables in P4 programs are accessed via table objects that must be initialized before performing any operations:

```
table1 = bfrt_info.table_get("Ingress.ipv4_host")
```

Adding an entry in the table

Entries are added to tables using the `entry_add` method, which requires a target device, a match key, and action data. Here's an example which adds an entry which maps IP address '5.5.5.7' to port 21:

```
ip_addr = socket.inet_aton('5.5.5.7')
ip_int = struct.unpack("!I", ip_addr)[0]
target= bfrt_client.Target(device_id=0,pipe_id=0xFFFF)

table1.entry_add(
    target,
    [table1.make_key([bfrt_client.KeyTuple('hdr.ipv4.dst_addr',
ip_int)]]),
    [table1.make_data([bfrt_client.DataTuple('port', 21)],
    'Ingress.send')]
)
```

Delete an entry in the table

Entries can be removed using the `entry_del` method by specifying a match key (e.g., removing the entry corresponding to IP address '5.5.5.7'):

```
ip_addr = socket.inet_aton('5.5.5.7')
ip_int = struct.unpack("!I", ip_addr)[0]
target = bfrt_client.Target(device_id=0, pipe_id=0xFFFF)
```



```

table1.entry_del(
    target,
    [table1.make_key([bfrt_client.KeyTuple('hdr.ipv4.dst_addr',
ip_int))]]
)

```

To delete all entries simultaneously, pass 'None' as the key parameter:

```

table1.entry_del(
    target,
    None #this will delete all entries
)

```

Update an entry in the table

Existing entries can be modified using the entry_mod method with the same key but updated action data. An example to update an existing entry (e.g., changing port from 21 to 22):

```

ip_addr = socket.inet_aton('5.5.5.7')
ip_int = struct.unpack("!I", ip_addr)[0]
target = bfrt_client.Target(device_id=0, pipe_id=0xFFFF)

table1.entry_mod(
    target,
    [table1.make_key([bfrt_client.KeyTuple('hdr.ipv4.dst_addr',
ip_int))]],
    [table1.make_data([bfrt_client.DataTuple('port', 22)],
    'Ingress.send')]
)

```

Reading (dump) current entries in the table

Table entries can be retrieved using the `entry_get` method. To read a specific entry:

```
ip_addr = socket.inet_aton('5.5.5.7')
ip_int = struct.unpack("!I", ip_addr)[0]
target = bfrt_client.Target(device_id=0, pipe_id=0xFFFF)

resp = table1.entry_get(
    target,
    [table1.make_key([bfrt_client.KeyTuple('hdr.ipv4.dst_addr',
ip_int))],
    {"from_hw": True}
)

for data, key in resp:          #print the entries
    print("Entry: Key:", key, "Data:", data)
```

To read all entries at once, pass 'None' as the key parameter:

```
resp = table1.entry_get(
    target,
    None,      #this will get all entries
    {"from_hw": True}
)
for data, key in resp:      #printing the entries
    print("Entry:")
    print("  Key:", key)
    print("  Data:", data)
```

Running bfrt_starter.py file

After making changes to the bfrt_starter.py file, we have to run it with the p4 code running in background.

To run simple_l3.p4 code in background we run the following command:

```
cd $SDE
./run_switchd.sh -p simple_l3 &
```

Exit bfshell by pressing enter and navigate to the directory where bfrt_starter.py exists and run the program:

```
python3 bfrt_starter.py
```

An Example to add an entry in the table:

First of all, edit the bfrt_starter.py to add an entry and view all entries afterwards in the following way and save the file:

ref: https://github.com/ashhraaffff/p4-experiments/blob/main/gRPC/bfrt_starter.py

```
##### You can now use BFRT CLIENT #####

table1 = bfrt_info.table_get("Ingress.ipv4_host")

ip_addr = socket.inet_aton('5.5.5.7')
ip_int = struct.unpack("!I", ip_addr)[0]
target= bfrt_client.Target(device_id=0,pipe_id=0xFFFF)

table1.entry_add(
    target,
    [table1.make_key([bfrt_client.KeyTuple('hdr.ipv4.dst_addr', ip_int)]),
     [table1.make_data([bfrt_client.DataTuple('port', 21)], 'Ingress.send')]]
)
print("entry added")
print()

print("Reading all entries:")
resp = table1.entry_get(
    target,
    None,
    {"from_hw": True}
)
for data, key in resp:
    print("Entry:")
    print("  Key:", key)
    print("  Data:", data)

##### FINALLY #####

# If you use SDE prior to 9.4.0, uncomment the line below
# interface._tear_down_stream()

print("The End")
```

Compile, build and load the p4 program using previously given steps and then run it in the background using this command:

```
cd $SDE
./run_switchd.sh -p simple_L3 &
```

Now run the bfrt_starter.py file using this command:

```
python3 bfrt_starter.py
```

You will be able to see that an entry has been added in the ipv4_host table:

```
Subscribe attempt #1
Subscribe response received 0
Connected to BF Runtime Server
Received simple_l3 on GetForwarding on client 0, device 0
The target runs the program simple_l3
Binding with p4_name simple_l3
Binding with p4_name simple_l3 successful!!
entry added

Reading all entries:
Entry:
  Key: {'hdr.ipv4.dst_addr': {'value': 84215047}}
  Data: {'port': 21, 'action_name': 'Ingress.send', 'is_default_entry': False}
The End
```

Packet Redirection Between Tofino Pipeline and Switch CPU

It is possible to send packets from the tofino pipeline to the switch os (CPU), and reinject the same packets from the CPU to the pipeline.(via devport 64 and 66).

The tofino pipeline (dataplane) is connected to the CPU via dedicated Ethernet MAC, and you can forward packets from the pipeline to the CPU on devport 66. These are 1G internal links to the switch OS and are always connected. To use these, we have to port enable them from ucli.

The switch has 32 physical qsfps ports and the 33rd (internal physical) port is the CPU port.

The steps are very similar to how you would set up and enable a physical port. (ref. Shivams report for a detailed description of UCLI)

```
bf-sde> pm
bf-sde.pm> show
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
PORT |MAC |D_P|P/PT|SPEED |FEC |AN|KR|RDY|ADM|OPR|LPBK |FRAMES RX |FRAMES TX |E
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
bf-sde.pm> port-add 33/- 1G NONE
bf-sde.pm> an-set 33/- 2
bf-sde.pm> port-enb 33/-
bf-sde.pm> show
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
PORT |MAC |D_P|P/PT|SPEED |FEC |AN|KR|RDY|ADM|OPR|LPBK |FRAMES RX |FRAMES TX |E
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
33/0 |32/0| 64|1/64|1G |NONE|Ds|Au|YES|ENB|UP | NONE | 1| 0|
33/1 |32/1| 65|1/65|1G |NONE|Ds|Au|YES|ENB|DWN| NONE | 0| 0|
33/2 |32/2| 66|1/66|1G |NONE|Ds|Au|YES|ENB|UP | NONE | 0| 0|
33/3 |32/3| 67|1/67|1G |NONE|Ds|Au|YES|ENB|DWN| NONE | 0| 0|
bf-sde.pm> show
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
PORT |MAC |D_P|P/PT|SPEED |FEC |AN|KR|RDY|ADM|OPR|LPBK |FRAMES RX |FRAMES TX |E
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
33/0 |32/0| 64|1/64|1G |NONE|Ds|Au|YES|ENB|UP | NONE | 3| 0|
33/1 |32/1| 65|1/65|1G |NONE|Ds|Au|YES|ENB|DWN| NONE | 0| 0|
33/2 |32/2| 66|1/66|1G |NONE|Ds|Au|YES|ENB|UP | NONE | 2| 2|
33/3 |32/3| 67|1/67|1G |NONE|Ds|Au|YES|ENB|DWN| NONE | 0| 0|
```

NOTE: The devport 66 can be used to both send and receive packets from the tofino pipeline, while devport 64 can only send packets from the CPU to the pipeline.

Sending packets to the CPU is just a forwarding decision at the match action table, just set the egress port to 66.

These CPU ports are visible as network devices/interfaces from ubuntu.
run > ip link show to see all the network devices/interfaces available

devport 66 maps to interface enp4s0f0
devport 64 maps to interface enp4s0f1

To view all the ports connected to the pipeline.

```
>bfshell
>ucli
>port_mgr
>ports
```

Output

```
Cpu port(s):
-+-+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
0| 1 | 0 | 64| 64|0| 0 |141|141| E | Up | 1g |none |10240| 0 | 0 | 0 | 0 | 0 | 0 |
0| 1 | 0 | 65| 64|1| 0 |142|142| E | Dn | 1g |none |10240| 0 | 0 | 0 | 0 | 0 | 0 |
0| 1 | 0 | 66| 64|2| 0 |140|140| E | Up | 1g |none |10240| 0 | 0 | 0 | 0 | 0 | 0 |
0| 1 | 0 | 67| 64|3| 0 |143|143| E | Dn | 1g |none |10240| 0 | 0 | 0 | 0 | 0 | 0 |
-+-+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
```

Self-learning switches

Self-learning switches dynamically populate their forwarding tables by observing incoming packets source and destination addresses, eliminating the need for manual configuration of match-action tables. The forwarding rules are drawn from a pre-existing database.

ref. https://github.com/ashhraaffff/p4-experiments/blob/main/self-learning-switches-v1/learning_switch_2.py

This can be accomplished by:

1. Updating the P4 code to forward the packets that do not find a match action to the CPU via one of the CPU ports (devport 66) by default. This is achieved by making the default action `send(CPU_PORT)`.

ref. https://github.com/ashhraaffff/p4-experiments/blob/main/self-learning-switches-v1/simple_l3.p4

```
19
20 const PortId_t CPU_PORT = 66;
21
```

```
table ipv4_host {
    key = { hdr.ipv4.dst_addr : exact; }
    actions = {
        send;
    }
    default_action = send(CPU_PORT);
    size = IPV4_HOST_TABLE_SIZE;
}

table ipv4_lpm {
    key      = { hdr.ipv4.dst_addr : lpm; }
    actions = { send; drop; }

    default_action = send(CPU_PORT);
    size           = IPV4_LPM_TABLE_SIZE;
}
```


2. The CPU-based control plane application sniffs on enp4s0f0 interface, which is mapped to this CPU port, capturing packets requiring address learning, using a python program.
3. Upon receiving the packet, the control plane extracts the relevant IP address and creates a corresponding forwarding table entry using gRPC, associating the target's destination address with the appropriate egress port.
4. The control plane then re-injects the original packet into the switch pipeline through the second CPU port (devport 64), which is mapped to the enp4s0f1 interface.
5. The reinjected packet traverses the pipeline again and is forwarded to Host B according to the newly created forwarding table entry.
6. Subsequent packets destined for the target will be forwarded directly by the switch using the established table entry, bypassing the CPU-based learning process.

PCIe Express CPU Port

“Pipe 0 is also connected to one additional 100Gbps Ethernet port, called the CPU Ethernet port.” (The connection link was 1G instead of 100Gbps). Pipe 2 is connected to a DMA packet interface, called the CPU PCIe port. The CPU PCIe port is always connected to a local CPU. The CPU Ethernet port is connected to a local CPU in some systems, or in others can be connected to a remote-control plane (e.g., an SDN controller). The control plane uses the APIs implemented in the driver software to configure everything through a PCI Express interface, often using DMA channels to increase the rate that large tables can be configured.

How to enable the DMA PCIe port for our switch?

The DMA PCIe port requires the kernel module `bf_kpkt` to be loaded, but in Barefoot SDE environments, `bf_kdrv` and `bf_kpkt` are typically mutually exclusive - they both register with the same name 'bf' in the kernel. So we have to first remove the base `kdrv` module and load the `kpkt` module

```
sudo rmmod bf_kdrv
//load kpkt
sudo $SDE_INSTALL/bin/bf_kpkt_mod_load $SDE_INSTALL
sudo lsmod | grep bf
//you should see bf_kpkt
```

The PCIe network interface is registered with the name `ens1`

`dmesg` output

```
[4988790.399966] bf_kdrv: module unloading ...
[4988790.400774] bf_kdrv: module unloaded successfully
[4988829.199522] bf_kpkt: Intel(R) Switch ASIC Linux Packet Driver -
version 1.0
[4988829.199529] bf_kpkt: Copyright (c) 2015-2022 Intel Corporation.
[4988829.199556] bf device class created
```

```
[4988829.200371] bf using 2 MSI irq from 65
[4988829.200536] BF allocating 2 MSI vectors from 65
[4988829.200541] bf 0000:06:00.0: bf device 0 registered with irq 65
[4988829.200546] bf probe ok
[4988829.205824] bf_kpkt: NAPI Net Driver, version 9.13.3-329-cpr
(01May2017)
[4988829.206851] bf_pci0 BF PCI-MAC is up
[4988829.206884] bf_kpkt kernel processing enabled for dev 0 subdev_id
0
[4988829.210615] bf 0000:06:00.0 ens1: renamed from bf_pci0
```

//going back to how it was

```
sudo rmmod bf_kpkt
cd ~/scripts/
source load_module.sh
```

NOTE: We learned that the bf_kpkt driver is a superset of the bf_kdrv (base bf driver) and supports all the functionality of the base driver.

Miscellaneous

`sudo lspci -vv -d 1d1c:` (colon included) shows you our switch information

<output looks like this>

```
06:00.0 Unassigned class [ff00]: Barefoot Networks, Inc. Tofino 1 (rev 10)
    Subsystem: Barefoot Networks, Inc. Tofino 1
    Physical Slot: 1
    Control: I/O- Mem+ BusMaster+ SpecCycle- MemWINV- VGASnoop- ParErr- Stepping-
<MAbort- >SERR- <PERR- INTx-
    Latency: 0, Cache Line Size: 32 bytes
    Interrupt: pin A routed to IRQ 65
    NUMA node: 0
    Region 0: Memory at f4000000 (64-bit, prefetchable) [size=64M]
    Region 2: Memory at f8004000 (64-bit, prefetchable) [size=16K]
    Region 4: Memory at f8000000 (64-bit, prefetchable) [size=16K]
    Capabilities: [80] Express (v2) Endpoint, MSI 00
    DevCap:      MaxPayload 512 bytes, PhantFunc 0, Latency L0s <2us, L1 <2us
```

`sudo ls -al /sys/class/net` shows all available network interfaces

1. Resource busy error (tmux)
2. pkill

Future work

- Figuring out if there is a way to piggy back some information from the p4 code (say the ingress port meta data) to the python code, for control plane management.
- Accessing table metadata using gRPC. Table entry hits, stale entries and hit timestamps. Using this to build the next version of self learning switches which automatically removes stale entries from time to time.